

Fine-Tune It the Way You Trained It: Field-Target Reinforcement Learning for One-Step Policies

Anonymous authors
Paper under double-blind review

Abstract

One-step generative policies distill multimodal demonstration data into a single network evaluation, but fine-tuning them with reinforcement learning is structurally awkward: their likelihoods are intractable, so policy-gradient ratios require injecting noise into the deployed policy, and backpropagating a learned critic through the policy ties the size of every update to the magnitude of the critic’s action-gradient — a channel through which an overfit critic drives the policy off the data manifold. In a controlled two-regime experiment, we show that this failure is catastrophic precisely when critic data is scarce relative to the action dimension, and that behavior-cloning penalties do not prevent it. It disappears when the update is instead expressed as bounded particle transport. Building on this, we propose Porygon: reinforcement fine-tuning of a one-step drifting policy with the same field-target regression primitive that pretrained it. A frozen base proposes action particles; a residual head is regressed onto targets displaced by two velocity fields — a clipped critic-gradient reward field and a restoring anchor. The clip acts pointwise in action space, so the policy can move only a bounded distance per update no matter how wrong the critic is. On robomimic, Porygon lifts a drifting base from 0.877 to 0.99 on can and from 0.382 to 0.91–0.93 on square — ahead of the backpropagated update at the matched budget on every seed, and matching the flow-matching + DICE-RL pipeline on both tasks despite a base less than half as strong on square. On the eight hardest LIBERO-90 tasks it gains +9.4 points on average with one shared recipe — improving on every task, by up to +24 — where the backpropagated actor gains +0.9 and falls below its base on the two tasks with the most headroom, and matches a critic-free policy-gradient baseline at equal interaction budget without the noise injection that baseline requires. Ablations over lower-resolution reward fields (top- k , exponentially tilted) map exactly when the critic’s gradient — versus only its values — should be trusted.

1 Introduction

One-step generative policies are an appealing endpoint for imitation learning: a drifting model (Deng et al., 2026) captures a multimodal demonstration distribution in a single network evaluation — diffusion-quality action distributions at deployment-friendly cost. It is trained by a simple primitive: repeatedly transport a population of its own samples along a velocity field toward the data, and regress the network onto the transported targets. This paper is about what that primitive buys after pretraining. Like any behavior-cloned policy, they plateau at the quality of their demonstrations, and the natural next step is reinforcement-learning fine-tuning with sparse task reward. That step, however, is structurally awkward for exactly this policy class.

The two standard fine-tuning routes both break on this policy class. Likelihood-ratio methods (PPO/GRPO-style) need $\log \pi(a \mid s)$, which a deterministic one-step noise-to-action map does not expose; the chain-MDP reformulations that rescue multi-step diffusion poli-

cies (Ren et al., 2025) have no analog when there is only one step. They can be applied only by changing the policy — injecting Gaussian noise around its output to define a surrogate likelihood (Zhang et al., 2026) — which alters the action distribution that is finally deployed. We use exactly that surrogate for our policy-gradient baseline, and it is a competitive one; the question this paper asks is what can be done without modifying the policy at all. The remaining route — learn a critic and backpropagate $-Q$ through the actor, as the state-of-the-art residual fine-tuner DICE-RL (Sun & Song, 2026) does — silently ties the size of every policy update to the magnitude of the critic’s action-gradient. A learned critic is trained to rank actions near its data; its gradient field off-manifold is unconstrained (Fujimoto et al., 2018; Jain et al., 2025). In a controlled toy (Section 3.3), this failure is total: with high-dimensional actions and scarce critic data, the backpropagated update drives true reward to zero while its internal Q -estimate rises by four orders of magnitude. Adding a BC penalty does not save it — a penalty in the loss must out-shout an unbounded gradient, and loses. Yet in the complementary regime, where the critic generalizes, the same gradient is the best available signal and value-only alternatives waste it. A useful update rule must be safe in the first regime without discarding the gradient in the second.

The lesson we draw is blunt: a learned critic is a judge, and handing a judge a compass — letting its gradient magnitude set the step — is what breaks. But discarding the compass everywhere, as critic-as-judge methods do (Peng et al., 2019; Wang et al., 2020), wastes the best signal available in the benign regime. The resolution is to change not the signal but the form of the update — and the policy class itself already contains the right form. A drifting model is pretrained by field-target regression: transport samples along a velocity field, regress the network onto the transported targets. Porygon fine-tunes with the very same primitive. We call the resulting method Porygon. A frozen base proposes action particles through a residual head; reward enters as a velocity field V_Q — by default the normalized critic gradient — which is clipped pointwise in action space and combined with a restoring anchor field before the residual is regressed onto the displaced targets. The trust region thus lives where the damage occurs: the policy provably moves at most δ per update at every particle, regardless of critic pathology, a guarantee parameter-space gradient clipping cannot give. No likelihood is ever needed, and in the small-step limit the update provably recovers the backpropagated direction (Proposition 1) — the safety is free.

Because reward enters only as a field, the critic can also be consumed at lower resolution when its gradients cannot be trusted: transport toward critic-ranked top- k particles, or toward the cloud’s own $\exp(Q/\tau)$ -tilted density. Our experiments treat this as an explicit dial and map its regimes. On robomimic, where the critic generalizes, the gradient field is the resolution to use: from a drifting base at 0.877, Porygon reaches 0.99 on can, and lifts square from 0.382 to 0.91–0.93 across seeds — ahead of the backpropagated control at the matched budget (0.87–0.91) and at parity with the flow-matching + DICE-RL state-of-the-art pipeline on both tasks, while the value-only sources trail far behind (~ 0.55 on square). On the eight hardest LIBERO-90 tasks, fine-tuned per task from one shared base with one shared recipe, Porygon gains +3.9 to +23.9 points (mean +9.4) where the backpropagated actor consuming the same critics gains +0.9 and loses ground on the two highest-headroom tasks. The ordering is unchanged when the base is swapped for the stronger flow-matching policy, and Porygon matches a critic-free policy-gradient baseline at equal budget without needing the noise injection that baseline requires. With a single critic shared across all eight tasks, no critic-based update rule — backpropagated or field-based, ours or the state of the art’s — improves on its own base, locating the binding constraint in per-task critic quality exactly as the toy predicts.

Our contributions:

- A field-target RL update for one-step policies (Section 4): reward, trust region, and anchoring expressed as composable velocity fields, with a pointwise action-space movement bound and a dead-zone restoring anchor; likelihood-free by construction and a strict generalization of the backpropagated residual actor.
- A controlled diagnosis of critic exploitation (Section 3.3): a two-regime toy showing the failure is real, regime-dependent, and not fixed by BC penalties — motivating trust regions in action space rather than loss space.

- Evidence that the form is what wins (Section 5): under matched budgets and one shared recipe, Porygon beats the backpropagated update on the same base with the same reward signal on both benchmarks and on both bases, lifts a weaker one-step base to match the stronger flow-matching + DICE-RL pipeline, and reaches parity with a critic-free policy-gradient baseline without modifying the deployed policy. Leave-one-out ablations isolate what pays: the action-space trust region and its dead-zone anchor, not the step size, and the critic’s gradient direction, not its ranking.

2 Related Work

RL fine-tuning of generative robot policies. Policy-gradient fine-tuning of vision-language-action models with sparse task reward is now standard for autoregressive token policies (Li et al., 2025), but transfers poorly to continuous generative heads: likelihood ratios are intractable for diffusion and flow policies, and workarounds either reformulate the denoising chain as an MDP (Ren et al., 2025; Black et al., 2024) — a construction with no analog for one-step generators — or avoid likelihoods entirely (Park et al., 2025; Zhang et al., 2026; Mark et al., 2024). Closest to our setting, DICE-RL (Sun & Song, 2026) fine-tunes a frozen one-step distillation of a flow-matching policy by training a residual head with a backpropagated critic and a filtered BC anchor; it is our primary baseline and the control for our actor-update ablation, since we keep its critic, residual parameterization, and data pipeline unchanged.

Critic-as-compass versus critic-as-judge. That a learned Q -function’s action-gradient cannot be trusted is a recurring diagnosis: overestimation and narrow spurious peaks in TD3 (Fujimoto et al., 2018), extrapolation error in BCQ (Fujimoto et al., 2019), and spurious local optima that make sample-and-select beat gradient ascent (Jain et al., 2025). One response treats the critic only as a judge: weighted regression and sample-based improvement (AWR, CRR, MPO, IDQL, QT-Opt) (Peng et al., 2019; Wang et al., 2020; Abdolmaleki et al., 2018; Hansen-Estruch et al., 2023; Kalashnikov et al., 2018) tilt the policy toward $\pi \cdot \exp(Q/\tau)$ using only Q -values. Our method spans this spectrum rather than picking a side: the update form is judge-safe (bounded transport toward targets), while the reward field inside it can still be the analytic gradient where it is trustworthy — and our ablations map exactly when each choice wins.

Particle transport as a policy update. Regressing a sampler onto displaced particles is the amortized-SVGD idea of Soft Q-Learning (Haarnoja et al., 2017), with kernel repulsion preserving diversity as in SVPG (Liu et al., 2017); drifting models (Deng et al., 2026) make the same operator the pretraining objective of a one-step generator. Drifting Field Policy (Koo et al., 2026) is the closest prior method: it also performs RL on a drifting backbone by drifting fresh policy samples toward critic-ranked top- K candidates. Porygon differs in three ways: we update a residual on a frozen base rather than the full policy, we transport with an explicit pointwise trust region and a restoring anchor field (the stability contract our toy shows is load-bearing), and our reward field is a dial — including the analytic $\nabla_a Q$ source that dominates when the critic generalizes — rather than a fixed ranking surrogate.

Residual RL and staying on-manifold. Learning a residual on a frozen base is a long-standing recipe for sample-efficient robot RL; DAWN (Ma et al., 2026) identifies the value cold-start and scale pathologies of residual critics that we also observe. The off-manifold drift our toy exhibits is the mechanism behind over-optimization and mode collapse in RL-finetuned generators (Black et al., 2024; Fan et al., 2023; Nakamoto et al., 2023), and forgetting in RL-finetuned language models is likewise governed by divergence from the base (Shenfeld et al., 2025) — which cannot even be measured without likelihoods, reinforcing update rules whose step size is bounded in action space by construction.

In sum, prior work either trusts the critic’s gradient and pays for it off-manifold, or discards it and pays on hard tasks; Porygon is, to our knowledge, the first fine-tuner for one-step

generative policies whose update form is safe under an arbitrary critic while still admitting the full first-order signal when it is available.

3 Problem Statement

3.1 Setup

We consider an episodic decision process with states s , action chunks $a \in \mathbb{R}^{H \times d}$ (H control steps of a d -dimensional action, treated as a single vector of dimension $S = Hd$), and sparse task-success reward. A one-step generative policy maps a state and a noise sample to an action in a single network evaluation,

$$a = \pi_0(s, z), \quad z \sim \mathcal{N}(0, I), \quad (1)$$

and is pretrained on demonstrations with a drifting-model objective (Deng et al., 2026): at each step, a population of policy samples is transported by a kernel velocity field toward the data distribution, and the network is regressed onto the transported targets. We write $\mathcal{V}(x \mid p^+, p^-)$ for this transport field acting on query particles x , attracted to particles of p^+ and repelled from particles of p^- (Section 4.2 recalls its exact form).

Following residual fine-tuning practice (Ma et al., 2026; Sun & Song, 2026), we freeze the base and learn a residual head on top of it,

$$a = \pi_0(s, z) + r_\theta(s, z), \quad (2)$$

together with an ensemble critic $Q_\phi(s, z, a)$ trained by temporal difference with a conservative (CQL-style) penalty (Kumar et al., 2020). The critic training is standard and identical for every method compared in this paper; the object of study is solely the actor update — how the policy consumes the critic.

3.2 Two structural failure modes

No likelihoods. Policy-gradient fine-tuners (PPO/GRPO-style, e.g. SimpleVLA-RL (Li et al., 2025)) require $\log \pi(a \mid s)$. A one-step generator defines its action distribution implicitly through the noise map Equation (2); its likelihood is intractable, and unlike multi-step diffusion policies there is no chain-MDP reformulation to fall back on (Ren et al., 2025; Park et al., 2025). The ratio-based toolbox becomes available only after modifying the policy — adding fixed-variance Gaussian noise to its output so that a surrogate log-density exists (Zhang et al., 2026) — so the deployed policy is no longer the deterministic one-step map that was pretrained. Our experiments include this surrogate as a baseline; the update we develop needs no such modification.

Critic exploitation through backpropagation. The remaining standard update — backpropagate $-Q$ through the actor, as in DDPG lineage methods and in DICE-RL’s residual actor (Sun & Song, 2026) — couples the size of the policy update to the magnitude of the critic’s action-gradient: $\nabla_\theta \mathbb{E}[-Q] = -\nabla_a Q \cdot \partial a / \partial \theta$. But a learned critic is trained only to rank actions near its data. Off-manifold, its gradient field is unconstrained — and can be steep and wrong at the same time (Fujimoto et al., 2018; 2019; Jain et al., 2025). A steep, wrong gradient marches the policy off the data manifold at maximum speed: the better the critic looks to the optimizer, the worse the true policy becomes.

3.3 A two-regime toy experiment

To see the failure cleanly we build the smallest setting that contains it: a bandit whose expert data has two modes, a one-step policy pretrained on that data with the drift operator, and a critic trained only on actions the expert actually takes. The critic is therefore accurate on the data and unconstrained everywhere else — the situation every offline-trained critic is in. We then fine-tune the same base with two update rules and vary one thing: the action dimension, which controls how much of the space the critic has never seen.

When the critic is reliable, follow it. With 2-D actions and dense critic data, backpropagation is the best update (0.79 true reward, versus 0.53 for a value-only field update and 0.50 for

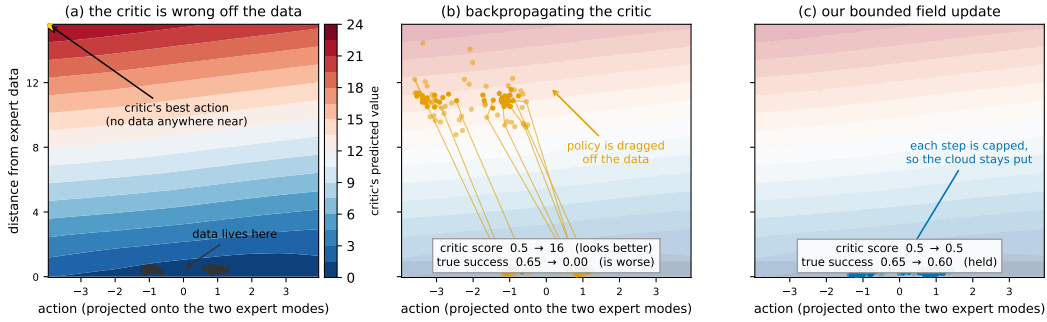


Figure 1: Why backpropagating a critic breaks, and what fixes it. Each panel shows the same action space, flattened to two axes: how far along the two expert modes an action lies (horizontal) and how far it is from any expert data (vertical, so the grey band at the bottom is where the demonstrations live). Colour is what the critic thinks an action is worth. (a) The critic is trained only on expert actions, so away from them it is free to invent value — its highest-scoring action (star) sits far from anything it has seen. (b) Backpropagating $-\hat{Q}$ through the policy sets each step’s size by the critic’s gradient, and the action cloud (three snapshots, connected) is pulled straight into that invented region: the critic’s score rises $32\times$ while the policy’s true success falls to zero. (c) Our update uses the same critic, but as a clipped velocity on each action plus a restoring anchor, so no update can move an action more than a fixed distance: the cloud stays on the data and success is preserved.

the base). Nothing is wrong with using a critic’s gradient when the critic knows what it is talking about.

When it is not, backpropagation is catastrophic — and a BC penalty does not save it. With 16-D actions and 60 critic training points, the same update drives true reward to zero while the critic’s own estimate of that policy rises by four orders of magnitude: the policy has walked into a region the critic scores highly and has never seen. Adding the standard behavior-cloning penalty changes nothing (0.00 as well). This is the diagnostic result of the paper: the penalty is a term in a loss, and it is competing against a gradient of unbounded magnitude. It loses. Figure 1 shows the geometry — the policy is not wandering, it is being pulled, quickly and directly, toward the critic’s hallucinated optimum.

The fix that follows. If the danger is that the update’s size is set by the critic’s gradient magnitude, then the cure is not a stronger penalty but a bound on how far the policy may move — and the bound must live in action space, where the damage happens, not in parameter space, where gradient clipping lives. Replacing the backpropagated update with one that transports each action particle by a clipped velocity does exactly this: in the same 16-D setting it holds the base’s performance (0.61 vs. base 0.65) and keeps every action on the data manifold, while preserving the action diversity the base was trained for (best-of-16 sampling reaches 0.77).

Two conclusions carry into the method. First, the choice is regime-dependent: the critic’s gradient is the best signal available when it generalizes and a liability when it does not, so a useful update must keep it without being steered by its magnitude. Second, what protects the policy is not which reward signal is used but the form of the update — a displacement bounded pointwise in action space, plus a restoring pull toward the base. In slogan form: keep the compass, but bolt it to a governor.

4 Method

Section 3.3 left us with a concrete requirement: use the critic’s gradient as a direction, but never let its magnitude decide how far the policy moves, and bound that movement where the damage happens — in action space. Porygon meets it by borrowing the operation that

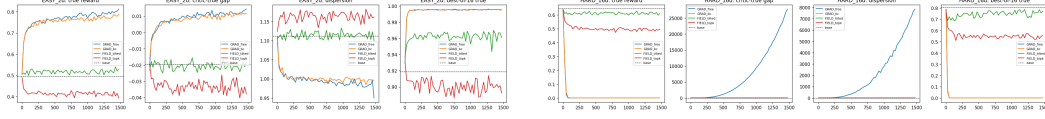


Figure 2: The two-regime toy (3 seeds). Left, easy regime (2-D actions, dense critic data): the critic’s gradient generalizes and backpropagation ($-Q$, with or without a BC anchor) is the best update. Right, hard regime (16-D actions, 60 critic points): the same updates collapse to zero true reward while their internal Q -estimate explodes — the BC anchor does not help — whereas the bounded field-target update stays on-manifold and holds the base policy. A useful fine-tuner must keep the gradient where it is trustworthy and survive where it is not.

[Figure reserved: method overview.] Left: the frozen base π_0 maps noise particles z_i to a cloud of actions; the residual r_θ shifts them. Middle: the three velocity fields at each particle — clipped critic gradient V_Q (with the top- k / tilted alternatives inset), dead-zone anchor V_{anc} — and the total-motion clip δ_{tot} . Right: the regression of r_θ onto the displaced targets; the same schematic as drifting pretraining with one extra field. To be drawn (TikZ or Figma).

Figure 3: Porygon overview: reinforcement fine-tuning as one more velocity field in the pretraining transport. [TODO: draw]

pretrained the policy in the first place: move action samples along a velocity field, then regress the network onto the moved samples. Fine-tuning becomes one more field added to that transport.

Two fields do all the work, and they are the paper’s main contribution. The reward field V_Q says which way each action should move to score better, and is clipped so no single update can be large. The anchor field V_{anc} pulls the policy back toward its base, but only once it has strayed beyond a set radius — inside that radius it is silent, so it never fights the reward field. Neither is a loss term: both are displacements applied to actions before the regression, which is exactly why the anchor cannot be out-shouted the way a behavior-cloning penalty was in Section 3.3. No likelihood is required, and no critic gradient is ever backpropagated into the actor.

4.1 The field-target update

One update works as follows. For each state in the batch we draw K noise samples and form the current action particles $a_i = \pi_0(s, z_i) + r_\theta(s, z_i)$, treated as constants (detached). Each particle is displaced by a combined velocity,

$$\tilde{r}_i = \text{sg}\left[r_i + \text{clip}_{\delta_Q}(V_Q(s, z_i, a_i)) + V_{\text{anc}}(r_i)\right], \quad \mathcal{L}_{\text{actor}} = \frac{1}{K} \sum_i \|r_\theta(s, z_i) - \tilde{r}_i\|^2, \quad (3)$$

where $\text{sg}[\cdot]$ is stop-gradient and $\text{clip}_\delta(v) = v \cdot \min(1, \delta/\|v\|)$ rescales a field vector to norm at most δ .

In words, one update does four things. (1) Propose: draw K noise samples and push them through the frozen base plus the current residual to get a cloud of candidate actions for this state. (2) Score and displace: ask the critic which direction improves each candidate, clip that direction to length δ_Q , and add the anchor’s pull if the residual has strayed past its radius; clip the resulting total displacement to δ_{tot} . (3) Freeze the targets: the displaced

actions are detached — they are data now, not a computation graph. (4) Regress: train the residual head to output those displaced targets. The critic is trained separately (Section 4.4); it is never differentiated through the actor, only queried.

Why this primitive, for this policy class. Field-target regression is not merely a safe update — it is the operation the network was shaped by. A step-distilled generator fine-tuned under a mismatched objective drifts from the distribution its one-step map encodes (Jiang et al., 2026; Yang et al., 2024); by keeping fine-tuning inside the pretraining operator family, Porygon changes only where the targets sit, not what kind of function the network is asked to represent. We refer to this as operator-consistent fine-tuning.

A pointwise trust region in action space. The clip in Equation (3) is the load-bearing difference from backpropagation. Under $\mathcal{L} = \mathbb{E}[-Q]$, the actor’s step scales with $\|\nabla_a Q\|$ — the critic-exploitation channel of Section 3.2. Under Equation (3), the policy (not the parameters) provably moves at most δ_{tot} per update at every particle, regardless of how steep or pathological the critic is. Gradient clipping in parameter space provides no such guarantee. This is what allows Porygon to run aggressive reward steps ($\eta_Q = 0.5$) that destabilize the backpropagated control (Section 5).

4.2 The reward field V_Q

The default (and headline) reward field is the normalized critic gradient evaluated at each particle,

$$V_Q^{\text{grad}}(s, z, a) = \eta_Q \frac{\nabla_a \bar{Q}_\phi(s, z, a)}{q_{\text{scale}}}, \quad q_{\text{scale}} = \mathbb{E}[|\bar{Q}_\phi|], \quad (4)$$

where $\bar{Q}_\phi$ is the conservative ensemble aggregate. This is deliberately the same scalar the DICE-RL actor optimizes: in the small-step, unclipped limit, minimizing Equation (3) recovers the backpropagated update direction exactly (Proposition 1), so any performance difference is attributable to the update form — the action-space trust region and target regression — not to a different reward signal.

The kernel transport operator. The lower-resolution reward fields below, the pretraining objective, and the distributional anchor variant all use the same drifting-model transport operator (Deng et al., 2026). Given query particles x , attractor particles p^+ (optionally weighted), and repulsor particles p^- , distances are normalized by a global scale and, for each kernel radius R in a small set, a soft mutual-matching affinity $A = \sqrt{\text{softmax}_{\text{row}}(-d/R) \odot \text{softmax}_{\text{col}}(-d/R)}$ converts matched attractors into pulls and matched repulsors into pushes; forces are summed over radii to give $\mathcal{V}(x | p^+, p^-)$ (full form in Appendix C). Intuitively, $\mathcal{V}$ moves a particle cloud so its density flows toward p^+ and away from p^- — it needs only samples of the target, never a density or a gradient.

Because V_Q enters Equation (3) only as a velocity, the critic can be consumed at lower resolution when its gradients are not trustworthy, using only Q -values:

- zeroth-order (top- k): rank the K particles by Q and set $V_Q = \mathcal{V}(a | \text{top-}k \text{ particles, all particles})$ — the pretraining kernel transport toward the critic’s preferred candidates;
- tilted: let every particle attract with weight $K \cdot \text{softmax}_i(\text{adv}_i/\tau)$, transporting the cloud toward its own $\exp(Q/\tau)$ -tilted density, the canonical improvement target of Peng et al. (2019); Abdolmaleki et al. (2018).

These are not competing methods but settings of a dial — how much of the critic’s local structure the update trusts: full first-order information, ranking only, or soft value weights. Section 5 maps the regimes: the gradient source dominates wherever the critic generalizes, while the value-only sources close the gap precisely in the weak-critic regime predicted by Section 3.3.

Algorithm 1 Porygon: one update (V_Q^{grad} variant)

Require: frozen base π_0 ; residual r_θ ; critic Q_ϕ ; steps $\eta_Q, \eta_{\text{anc}}$; clips $\delta_Q, \delta_{\text{tot}}$; dead zone ρ ; particles K

- 1: sample batch of states s ; draw $z_i \sim \mathcal{N}(0, I)$, $a_i \leftarrow \pi_0(s, z_i) + r_\theta(s, z_i)$ $\triangleright$ no grad
- 2: $V_i \leftarrow \text{clip}_{\delta_Q}(\eta_Q \nabla_a \bar{Q}_\phi(s, z_i, a_i) / q_{\text{scale}})$ $\triangleright$ or top- k / tilted kernel field
- 3: $V_i \leftarrow V_i - \eta_{\text{anc}}(1 - \rho / \|r_i\|)_+ r_i$ $\triangleright$ dead-zone anchor, Equation (5)
- 4: $\tilde{r}_i \leftarrow \text{sg}[r_i + \text{clip}_{\delta_{\text{tot}}}(V_i)]$ $\triangleright$ policy moves $\leq \delta_{\text{tot}}$, always
- 5: $\theta \leftarrow \theta - \alpha \nabla_\theta \frac{1}{K} \sum_i \|r_\theta(s, z_i) - \tilde{r}_i\|^2$
- 6: update ϕ : ensemble TD + CQL penalty $\triangleright$ unchanged from Sun & Song (2026)

4.3 The anchor field

The kernel reward fields vanish beyond their largest radius and the gradient field carries no notion of the data manifold, so Equation (3) alone would let the residual random-walk. The anchor is a restoring field with a dead zone,

$$V_{\text{anc}}(r) = -\eta_{\text{anc}} \left(1 - \frac{\rho}{\|r\|}\right)_+ r, \quad (5)$$

which is inert inside the trust radius ρ and pulls back only the excess residual norm beyond it. Unlike a $\lambda\|r\|^2$ penalty — which competes with an unbounded gradient and loses (Section 3.3) — Equation (5) composes with the clipped reward field as a velocity, so the equilibrium residual scale is set geometrically ($\|r\| \approx \rho + \delta_Q / \eta_{\text{anc}}$) rather than by a loss-weight arms race. Setting $\rho = 0$ recovers the pointwise pull $-\eta_{\text{anc}}r$, whose fixed point reproduces the DICE-RL anchor; the dead zone matters exactly when the reward field is weak (Section 5, LIBERO).

4.4 Relation to the backpropagated actor; critic

Proposition 1 (Small-step equivalence). At the current parameters, the gradient of Equation (3) is $\nabla_\theta \mathcal{L}_{\text{actor}} \propto -(\text{clip}_{\delta_Q}(V_Q) + V_{\text{anc}})^\top \partial r_\theta / \partial \theta$. With $V_Q = V_Q^{\text{grad}}$, no clipping ($\delta_Q, \delta_{\text{tot}} \rightarrow \infty$) and $\rho = 0$, this is the DICE-RL residual actor gradient with anchor weight η_{anc} .

Porygon is therefore a strict generalization of its control: the new degrees of freedom are the pointwise action-space clip, the dead-zone anchor, and the resolution dial on V_Q — each of which we ablate.

Remark (not just clipped gradient ascent). One might object that Equation (3) with V_Q^{grad} is “DDPG with clipping.” The distinction is where the bound lives. Clipping ∇_θ bounds a parameter step, whose effect on the policy output is unknown and state-dependent; Equation (3) bounds the policy’s motion at every particle, which is the quantity that off-manifold failure depends on (Section 3.3). Moreover, only the regression form makes the two other components expressible at all: a dead-zone anchor and a value-only V_Q have no loss-gradient counterpart. The ablations in Section 5.4 test each distinction in isolation. The critic is unchanged from DICE-RL: ensemble TD with a CQL-style penalty pushing Q down at policy actions and up at data actions; field construction never leaks gradients into the critic (gradients w.r.t. actions only). Algorithm 1 summarizes one joint update.

5 Experiments

Our experiments answer three questions. Q1: Does the field-target form beat backpropagation given the same reward signal and budget? Q2: Can a one-step drifting base fine-tuned with Porygon overtake the stronger flow-matching pipeline? Q3: When should the reward field use the critic’s gradient versus only its values — does the regime picture of Section 3.3 hold on real benchmarks?

Table 1: robomimic success rate (300-episode evaluations, last-3-checkpoint average, per seed). Bold marks the best value in each column (ties within 0.005). “—” = matched-budget run still in progress at submission-draft time; no provisional numbers are shown.

method	can			square		
	s42	s43	s44	s42	s43	s44
drift base (no RL)		0.877			0.382	
backprop actor (same base)	—	—	0.940	0.870	0.909	0.861
FM + DICE-RL	0.989	0.979	0.986	0.934	0.923	0.930
Porygon (top- k)	0.924	0.920	0.910	0.562	0.525	0.562
Porygon (tilted)	0.949	0.947	0.919	0.578	0.584	0.560
Porygon	0.988	0.993	0.978	0.912	0.930	0.929

5.1 Setup

Benchmarks. (i) robomimic can and square (state-based; 300-episode evaluations, average of the last three checkpoints; 3 seeds), run inside the official DICE-RL harness with only the actor update replaced — critic, replay, demonstrations, and evaluation protocol are byte-identical to the baseline. (ii) The eight hardest tasks of LIBERO-90 (multitask drifting base; 100 RL iterations; final numbers use powered evaluation: 100 rollouts $\times$ 3 evaluation seeds). [TODO: appendix table: env details, horizons, demo counts, budgets.]

Compared methods. Each comparison is chosen to isolate one of the three questions; every method on the drifting base shares the residual parameterization, critic, data, and budget, so only the actor update varies.

- Porygon: Equation (3) with the gradient reward field V_Q^{grad} .
- Backprop actor (the controlled comparison for Q1): the DICE-RL residual actor (Sun & Song, 2026) — backpropagated $-Q$ with a filtered BC penalty — run on our drifting base. It consumes the identical critic through the identical residual head, so any gap to Porygon is attributable to the update form, not to the base, the reward signal, or the data.
- FM + DICE-RL (the external reference for Q2): the unmodified state-of-the-art pipeline on its own flow-matching base (Sun & Song, 2026). It answers whether the one-step base plus Porygon competes with the strongest published system, not merely with its own control.
- Porygon (top- k) and Porygon (tilted) (resolution ablations for Q3, not baselines): the same update with the reward field built from the critic’s rankings (transport toward the top- k particles) or its soft values ($\exp(Q/\tau)$ -weighted self-attraction). Each deletes the critic’s gradient direction while changing nothing else, measuring how much of Porygon’s gain flows through that one channel — and, in the weak-critic LIBERO regime, whether the gradient is worth consuming at all.

Hyperparameters are family-shared across Porygon and its two resolution variants ($\eta_Q=0.5$, $\delta_{\text{tot}}=0.15$, $\eta_{\text{anc}}=1.0$, $\tau=1$, $k=4$; dead zone $\rho=0.05$ on LIBERO) so that comparisons isolate the update rule rather than tuning effort.

5.2 Robomimic: the gradient regime

Table 1 gives the per-seed results, and Figure 4 the corresponding learning curves. On can, fine-tuning saturates the task: Porygon reaches 0.978–0.993 from a base at 0.877, the flow-matching pipeline reaches 0.979–0.989, and the backprop actor trails slightly at 0.940 — the can task chiefly certifies that the field update preserves (and slightly extends) the base’s competence at the ceiling. The discriminative task is square, where the drifting base solves only 0.382 of episodes; Porygon adds 55 points, reaching 0.91–0.93 on every seed. The backprop actor — same base, same critic, same data, same budget — reaches 0.86–0.91 (Q1): given the identical reward signal, the field-target form extracts strictly more progress

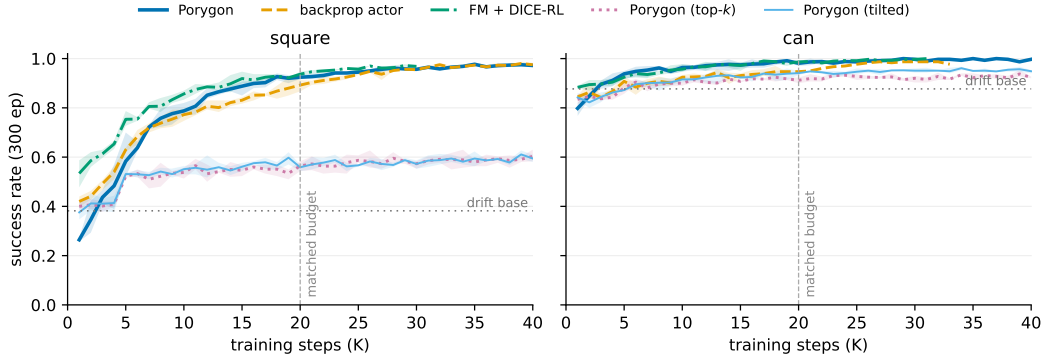


Figure 4: robomimic learning curves (300-episode evaluations every 1K steps; mean and min-max band over seeds; dashed vertical line marks the matched 20K budget). On square, Porygon overtakes the backprop actor by 7K steps and holds a 4–7-point lead through the matched budget; the value-only variants (top- k , tilted) flatline near 0.55. On can, Porygon and the flow-matching pipeline both approach the ceiling (≥ 0.98). [TODO: regenerate with the s44 and can-baseline reruns when they finish.]

at the matched budget. Against the flow-matching pipeline, Porygon reaches parity on both tasks (0.91–0.93 vs. 0.92–0.93 on square; 0.99 vs. 0.99 on can) — but from a base less than half as strong on square (Q2): the safer update recovers the headroom the weaker base gives up.

Why does the form win? The trust region converts critic error from a stability threat into bounded noise: with policy motion capped at δ_{tot} per update, Porygon tolerates a reward step ($\eta_Q=0.5$) far above what the backprop actor can survive, and simply takes more, safer steps in the directions where the critic is consistent. The backprop actor must instead keep its effective step small enough for its worst critic errors, and it pays in speed: Porygon reaches 0.91 on every seed by 15K steps — a level the backprop actor needs another 5–10K steps to reach. Its matched-budget endpoint trails Porygon’s on every seed, and the gap closes only near 40K, where both converge to 0.97–0.99. The form’s advantage on this benchmark is therefore a faster, never-worse path to the same asymptote — exactly what a bounded-step update should buy [TODO: quantify further with the stability-window ablation (Section 5.4, iii)].

The dial answers Q3 in its first regime: with a critic that generalizes (robomimic critics see dense, single-task data), the value-only reward fields preserve can (0.91–0.95) but collapse on square (0.53–0.58 versus Porygon’s 0.91–0.93), despite verified-healthy residuals and longer training. Deleting the gradient direction throws away exactly the signal that solves the harder task — the real-benchmark mirror of the toy’s easy regime, where the compass wins.

5.3 LIBERO hard-8: the weak-critic regime

On the eight hardest tasks of LIBERO-90 we deploy Porygon as a routing system: one frozen multitask drifting base, one small per-task residual head and critic fine-tuned by Porygon on each task’s own interactions, and task-ID routing at deployment — the architecture every published LIBERO RL fine-tuner effectively uses. One recipe is shared across all tasks with no per-task tuning ($\eta_Q=1.0$, dead-zone anchor with a dimension-independent rms radius $\rho=0.05$, 3-step returns, 40 iterations of 16 episodes).

Table 2 gives the verdict across all eight tasks: Porygon gains +3.9 to +23.9 points per task (mean +9.4 at $\eta_Q=2$, +7.7 at $\eta_Q=1$), improving on the base for every task at both settings — on the highest-headroom task it reaches 0.781, above the best previously recorded result on that task by any method — while the backpropagated actor consuming the same per-task critics averages +0.9, going below base on the two highest-headroom tasks. The

Table 2: LIBERO hard-8, per-task fine-tuning from the shared drifting base (success under powered evaluation: 100 rollouts $\times$ 3 evaluation seeds; Porygon rows average the last three training checkpoints, and two training seeds on t32/t53/t65). One recipe across all tasks. Across the five tasks trained with two seeds the spread is 0.000, 0.003, 0.004, 0.019 and 0.089 (t75, t73, t53, t32, t65): negligible except on the highest-headroom task, so on all tasks but t65 differences of a few points are meaningful, while on t65 only gaps above ~ 9 points are. “—” = run still in progress at draft time.

	t8	t21	t32	t53	t65	t73	t75	t81	mean
drift base (no RL)	0.818	0.690	0.610	0.785	0.573	0.648	0.578	0.588	0.661
backprop actor	0.820	0.703	0.587	0.800	0.550	0.663	0.583	0.653	0.670
Porygon ($\eta_Q=1$)	0.831	0.737	0.694	0.842	0.826	0.710	0.657	0.607	0.738
Porygon ($\eta_Q=2$)	0.857	0.752	0.729	0.853	0.812	0.752	0.648	0.635	0.755
same protocol, flow-matching base (4 tasks measured)									
FM base (no RL)	—	—	0.690	0.917	0.787	—	0.620	—	—
FM + DICE-RL	—	—	0.682	0.915	0.809	—	0.592	—	—
FM + GRPO	—	—	0.700	0.897	0.883	—	0.603	—	—
FM + Porygon	—	—	0.738	0.897	0.878	—	0.608	—	—

robomimic ordering (Q1) transfers intact to the harder benchmark: identical reward signal, and only the field-target form converts it into gains without ever falling below the base. Two implementation findings were load-bearing and are ablated in Section 5.4: the anchor’s dead zone must be defined in dimension-independent rms units (the vector-norm form silently re-activates the anchor at LIBERO’s $S=112$ chunk dimension and freezes the residual), and the weaker critic gradient ($\|\nabla_a Q\|$ roughly $4\times$ smaller than robomimic’s) requires the doubled reward step the trust region makes safe. Sweeping the reward step across all eight tasks confirms this at the table level: $\eta_Q=2$ improves the mean from 0.738 to 0.755 and helps on six of eight tasks (the two exceptions, -0.014 and -0.009 , are inside the seed spread), and the benefit is uncorrelated with the critic’s action-gradient magnitude — the task with the deadest critic (t81, $\|\nabla_a Q\|=0.007$) gains $+2.8$, as much as the strongest-gradient tasks. The reward step therefore acts mainly by reducing the anchor’s drag rather than by exploiting more first-order signal, and the trust region is what keeps the larger step safe: no setting on any task falls below its base.

The flow-matching rows settle Q1 independently of the base. On its own, stronger base the state-of-the-art pipeline extracts essentially nothing per task: FM + DICE-RL scores -0.8 , -0.2 , $+2.2$ and -2.8 points relative to the FM base on t32, t53, t65 and t75 (mean -0.4), the same near-zero extraction its actor shows on the drifting base ($+0.9$). Swapping only the actor update helps on three of the four tasks: Porygon on the identical flow-matching base and budget beats FM + DICE-RL by $+5.6$ (t32), $+6.9$ (t65) and $+1.6$ (t75), reaching 0.738 and 0.878 on the first two — $+4.8$ and $+9.1$ over that base. The exception is instructive: on t53 the flow-matching base already scores 0.917 and neither update improves it (-1.8 for Porygon, -0.2 for DICE-RL). Indeed the never-below-base behaviour Porygon shows on all eight drifting-base tasks does not carry over to this base: on t53 and t75 it ends 2.0 and 1.2 points below its starting point. Fine-tuning a base that is already near its ceiling is a different problem from lifting a weak one, and our trust region is calibrated for the latter.

Adding the critic-free policy-gradient baseline at the same budget completes the picture. On the one task with real headroom from the flow-matching base, GRPO and Porygon extract essentially the same amount (0.883 and 0.878 on t65, both $\approx +9.5$) while the backpropagated actor manages $+2.2$; on t32 Porygon leads (0.738 vs. 0.700); on the two tasks where no method gains, all three sit together. The separation on this benchmark is therefore not Porygon against the world but update form against backpropagation-through-the-critic: the field update matches what a policy-gradient method extracts, without requiring the noise-injection surrogate that makes likelihood ratios available for a one-step generator in the first place. The field-target form, not the base and not the reward signal, is what converts critic signal into success. It also puts the Q2 comparison in perspective: Porygon on the weak one-step base reaches 0.781 on t65 against the flow-matching pipeline’s 0.809 — within three

Table 3: LIBERO hard-8 mean success under powered evaluation (100 rollouts $\times$ 3 evaluation seeds per cell; one column per training seed; 100 RL iterations). Neither base is moved by any fine-tuner: the weak multitask critic supports no extractable gain, for backpropagation and field transport alike. “—” = evaluation still running.

method	s10000	s10001	s10002
drift base (no RL)	0.671 ± 0.014		
backprop actor (same base)	0.657	0.655	0.668
Porygon (top- k)	—	0.684	—
Porygon (tilted)	0.677	—	0.672
Porygon	0.655	—	0.659
flow-matching base (no RL)	0.738 ± 0.022		
FM + DICE-RL	0.756	0.735	0.737

[Figure reserved: two ablation panels.] Left — stability window: final square success vs. reward step η_Q for Porygon and for the backpropagated control (A + gradient clipping); the claim is a wide flat optimum for the field form vs. a narrow peak for backpropagation. Right — dispersion retention: action dispersion and best-of- N curve before/after fine-tuning for Porygon vs. A.

Figure 5: Ablations: where the trust region and update form pay. [TODO: runs in progress]

points, from a base 21 points lower — and exceeds it on t75 (0.657 vs. 0.592) and on t32 once the reward step is matched to that task’s stronger critic gradient (0.729 vs. 0.682). [TODO: per-task figure; remaining FM cells and matched-budget FM + GRPO.]

Why not one shared critic? The multitask control. The alternative to routing is a single policy and critic trained on all eight tasks jointly. This setting flips the critic’s situation: our diagnostics show the shared critic’s action-gradient collapses on most tasks (per-task $\|\nabla_a Q\|$ spanning 0.003 to 0.55), and the toy predicts the resolution dial should compress.

The powered numbers (Table 3) deliver a verdict that noisy small-sample readouts had obscured: nothing wins here. On the drifting base, the backprop actor (0.655–0.668), Porygon (0.655–0.659), and every resolution variant (0.66–0.70) all sit within noise of the untuned base (0.671 ± 0.014). The flow-matching pipeline looks stronger in absolute terms (0.735–0.756) — but its own base already scores 0.738 ± 0.022 , so DICE-RL likewise extracts essentially nothing; its apparent lead on this benchmark is inherited entirely from pretraining. Three honest conclusions follow. First, the weak-critic regime is a no-gain regime for every update rule we tested: when the shared critic’s action-signal is this poor, neither backpropagating it nor transporting along it, at any resolution, converts it into task success. Second, the dial compresses exactly as the toy predicts — gradient, ranking, and soft-value sources become indistinguishable — though in the starkest possible form. Third, every drift-base arm preserves the base within noise: the failure mode of Figure 1 does not appear even for the backprop actor, consistent with the diagnosis that this critic is action-blind rather than action-exploitable — there is no steep hallucinated gradient to ride. Small-sample evaluations told a different, wrong story (single-seed 20-episode endpoints ranged 0.63–0.78 across these same arms); powered evaluation collapses that spread to the base value. [TODO: per-task breakdown figure (scripts/plot_libero_pertask.py) once the remaining seed evaluations land; include the guarded BC-filter interaction in the appendix.]

Table 4: LIBERO ablations: leave-one-out on the frozen recipe, two tasks, powered evaluation (last-3 checkpoints). Every row changes exactly one component. Δ is relative to the full recipe.

variant	t65 (base 0.573)		t32 (base 0.610)	
	success	Δ	success	Δ
full recipe	0.781	—	0.684	—
– dimension-independent anchor (vector-norm ρ)	0.662	−0.119	0.655	−0.029
– dead zone ($\rho=0$)	0.549	−0.232	—	—
– doubled reward step ($\eta_Q=0.5$)	0.769	−0.012	0.615	−0.069
– 3-step returns ($n=1$)	0.782	+0.001	0.512	−0.172
wider trust region ($\delta_{\text{tot}}=0.25$)	—	—	0.689	+0.005
stronger reward step ($\eta_Q=2.0$)	—	—	0.729	+0.045
reward field $\rightarrow$ top- k (no gradient)	0.577	−0.204	—	—
reward field $\rightarrow$ tilted (no gradient)	0.697	−0.084	—	—

5.4 Ablations: which component pays where

Table 4 isolates each component. Three findings matter. First, the anchor geometry is load-bearing: removing the dead zone entirely ($\rho=0$) drops t65 to 0.549, below the untuned base — the un-gated restoring pull erases the residual, the quantitative version of the toy’s “penalties lose to fields” lesson — and measuring the dead-zone radius in vector-norm rather than dimension-independent units costs 11.9 points, because at LIBERO’s $S=112$ chunk dimension the base residual already exceeds ρ at initialisation, so the anchor is active from the first update. Second, deleting the critic’s gradient direction is fatal here too: top- k transport falls to 0.577 (base level) and tilted transport to 0.697, mirroring the robomimic square result and answering Q3 on both benchmarks. Third, the step-size and credit-horizon components are task-dependent in exactly the way the critic diagnostics predict: on t32, whose critic carries a strong action-gradient ($\|\nabla_a Q\| \approx 0.55$), 3-step returns are worth 17 points and the doubled step 7; on t65, whose gradient is an order of magnitude weaker (≈ 0.035), both are within noise. Pushing the reward step further on that task ($\eta_Q=2.0$) adds another 4.5 points (0.729), while widening the trust region alone ($\delta_{\text{tot}}=0.25$) changes nothing (+0.5): it is the reward step, not the movement bound, that the strong-gradient task wants. What a bounded-step field update needs is therefore not one universal step size but a step matched to the signal the critic actually carries — which the trust region is what makes safe to raise.

[TODO: Remaining ablation plots (Figure 5); robomimic-side items below: (i) trust-region locus — the backprop actor + parameter-space gradient clipping vs. B’s action-space clip at equal effective step (isolates the central claim); (ii) anchor geometry — $\rho=0$ silently erases the residual under weak reward fields (measured residual RMS collapses $\sim 5\times$ and performance reverts to base-plus-selection), the quantitative version of the toy’s “penalties lose to fields” lesson; (iii) step/clip sweep — the 21-configuration square sweep behind the family recipe: stability window of η_Q under Equation (3) vs. backpropagation; (iv) dispersion retention — action dispersion and best-of- N gains after fine-tuning, vs. control (does the field update preserve the multimodality the base was trained for?).]

6 Conclusion

We showed that the natural update primitives for RL fine-tuning of one-step generative policies are, respectively, unavailable without modifying the policy and unsafe when the critic is imperfect, and that the policy class’s own pretraining primitive — field-target regression — provides the missing update rule. Porygon expresses reward, trust region, and anchoring as composable velocity fields over action particles, provably bounds policy movement pointwise in action space, and recovers the backpropagated direction in the small-step limit.

The experiments locate the advantage precisely. Given the same base, critic, data and budget, the field-target form extracts far more than backpropagating that critic: +55 points on robomimic square, and +9.4 versus +0.9 on average across the eight hardest LIBERO-90 tasks, where the backpropagated actor falls below its base on the two highest-headroom tasks and Porygon improves on every one. The same ordering holds when the base is swapped for the stronger flow-matching policy, so the effect is attributable to the update rule rather than to the starting point. Against a critic-free policy-gradient baseline at matched interaction budget, Porygon is at parity (+10.1 versus +9.5 per-task mean on LIBERO; 0.878 versus 0.883 on the flow-matching base’s one high-headroom task) — while requiring none of the noise injection that makes likelihood ratios definable for a deterministic one-step generator in the first place. Ablations confirm the mechanism rather than the hyperparameters: removing the dead-zone anchor drops performance below the untuned base, deleting the critic’s gradient direction collapses gains to base level on both benchmarks, and the reward step can be doubled safely — the trust region, not a small step size, is what buys the stability.

Limitations. The gains track how much a critic actually knows. When a single critic is shared across all eight LIBERO tasks its action-gradient collapses and no critic-based update — ours or the state of the art’s — improves on its base; when a base is already near its ceiling, Porygon can end slightly below it (−2.0 and −1.2 on two flow-matching tasks), since our trust region is calibrated for lifting weak bases rather than for refining strong ones. Our critic-trust diagnosis is also post hoc: choosing the reward-field resolution online from critic-trust signals is the natural next step, as is testing whether the reward step should adapt per task. Finally, all results are state-based simulation on a single embodiment. The broader principle — fine-tune a policy with the operator that pretrained it — applies to any generative policy class trained by target regression, and testing it beyond drifting models is the direction we find most promising.

Reproducibility statement

[TODO: Code release (training framework + official-harness implant), exact configs (family-shared recipe), seeds, eval protocols (300-ep robomimic; 100×3 powered LIBERO), compute (single L40S/A100 per run, 8–20h), toy experiment script. All numbers in Table 1 are averages, never best-of.]

References

- Abbas Abdolmaleki, Jost Tobias Springenberg, Yuval Tassa, Remi Munos, Nicolas Heess, and Martin Riedmiller. Maximum a posteriori policy optimisation. In International Conference on Learning Representations (ICLR), 2018.
- Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. In International Conference on Learning Representations (ICLR), 2024. arXiv:2305.13301.
- Mingyang Deng, He Li, Tianhong Li, Yilun Du, and Kaiming He. Generative modeling via drifting, 2026. URL <https://arxiv.org/abs/2602.04770>.
- Ying Fan, Olivia Watkins, Yuqing Du, Hao Liu, Moonkyung Ryu, Craig Boutilier, Pieter Abbeel, Mohammad Ghavamzadeh, Kangwook Lee, and Kimin Lee. DPOK: Reinforcement learning for fine-tuning text-to-image diffusion models. In Advances in Neural Information Processing Systems (NeurIPS), 2023.
- Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In International Conference on Machine Learning (ICML), 2018.
- Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In International Conference on Machine Learning (ICML), 2019.

- Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In International Conference on Machine Learning (ICML), 2017.
- Philippe Hansen-Estruch, Ilya Kostrikov, Michael Janner, Jakub Grudzien Kuba, and Sergey Levine. IDQL: Implicit Q-learning as an actor-critic method with diffusion policies. arXiv preprint arXiv:2304.10573, 2023.
- Ayush Jain, Norio Kosaka, Xinhu Li, Kyung-Min Kim, Erdem Bıyık, and Joseph J. Lim. Mitigating suboptimality of deterministic policy gradients in complex q-functions, 2025. URL <https://arxiv.org/abs/2410.11833>.
- Dengyang Jiang, Xin Jin, Dongyang Liu, Zanyi Wang, Mingzhe Zheng, Ruoyi Du, Xiangpeng Yang, Qilong Wu, Zhen Li, Peng Gao, Harry Yang, and Steven Hoi. D-opsd: On-policy self-distillation for continuously tuning step-distilled diffusion models, 2026. URL <https://arxiv.org/abs/2605.05204>.
- Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, and Sergey Levine. QT-Opt: Scalable deep reinforcement learning for vision-based robotic manipulation. In Conference on Robot Learning (CoRL), 2018.
- Juil Koo, Mingue Park, Jiwon Choi, Yunhong Min, and Minhyuk Sung. Drifting field policy: A one-step generative policy via wasserstein gradient flow, 2026. URL <https://arxiv.org/abs/2605.07727>.
- Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline reinforcement learning. In Advances in Neural Information Processing Systems (NeurIPS), 2020.
- Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, Dehui Wang, Dingxiang Luo, Yuchen Fan, Youbang Sun, Jia Zeng, Jiangmiao Pang, Shanghang Zhang, Yu Wang, Yao Mu, Bowen Zhou, and Ning Ding. Simplevla-rl: Scaling vla training via reinforcement learning, 2025. URL <https://arxiv.org/abs/2509.09674>.
- Yang Liu, Prajit Ramachandran, Qiang Liu, and Jian Peng. Stein variational policy gradient. arXiv preprint arXiv:1704.02399, 2017.
- Guozheng Ma, Lu Li, Haoyu Wang, Zixuan Liu, Pierre-Luc Bacon, and Dacheng Tao. What makes value learning efficient in residual reinforcement learning?, 2026. URL <https://arxiv.org/abs/2602.10539>.
- Max Sobol Mark, Tian Gao, Georgia Gabriela Sampaio, Mohan Kumar Srirama, Archit Sharma, Chelsea Finn, and Aviral Kumar. Policy agnostic rl: Offline rl and online rl fine-tuning of any class and backbone, 2024. URL <https://arxiv.org/abs/2412.06685>.
- Mitsuhiko Nakamoto, Yuexiang Zhai, Anikait Singh, Max Sobol Mark, Yi Ma, Chelsea Finn, Aviral Kumar, and Sergey Levine. Cal-QL: Calibrated offline RL pre-training for efficient online fine-tuning. In Advances in Neural Information Processing Systems (NeurIPS), 2023.
- Seohong Park, Qiyang Li, and Sergey Levine. Flow Q-learning. arXiv preprint arXiv:2502.02538, 2025.
- Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. arXiv preprint arXiv:1910.00177, 2019.
- Allen Z. Ren, Justin Lidard, Lars L. Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. Diffusion policy optimization. In International Conference on Learning Representations (ICLR), 2025. arXiv:2409.00588.

Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. RL’s razor: Why online reinforcement learning forgets less, 2025. URL <https://arxiv.org/abs/2509.04259>.

Zhanyi Sun and Shuran Song. From prior to pro: Efficient skill mastery via distribution contractive rl finetuning, 2026. URL <https://arxiv.org/abs/2603.10263>.

Ziyu Wang, Alexander Novikov, Konrad Zolna, Josh Merel, Jost Tobias Springenberg, Scott Reed, Bobak Shahriari, Noah Siegel, Caglar Gulcehre, Nicolas Heess, and Nando de Freitas. Critic regularized regression. In Advances in Neural Information Processing Systems (NeurIPS), 2020.

Zhaorui Yang, Tianyu Pang, Haozhe Feng, Han Wang, Wei Chen, Minfeng Zhu, and Qian Liu. Self-distillation bridges distribution gap in language model fine-tuning. arXiv preprint arXiv:2402.13669, 2024.

Tonghe Zhang, Chao Yu, Sichang Su, and Yu Wang. Reinform: Fine-tuning flow matching policy with online reinforcement learning, 2026. URL <https://arxiv.org/abs/2505.22094>.

A Additional results

[TODO: Full seed tables, training curves, hard-8 per-task breakdown.]

B Hyperparameters

[TODO: Family-shared recipe table; official DICE-RL hypers; sweep grids.]

C The kernel transport operator

[TODO: Full form of $\mathcal{V}$: global distance scale, per-radius softmax-softmax affinity, contested-attractor weighting, self-exclusion, per-radius normalization on/off (restoring vs. constant-pull), radii $\{0.02, 0.05, 0.2\}$. Port from `drift_field.py` docstrings.]

D Toy two-regime construction

[TODO: Details of the hard-critic/easy-critic toy; failure of compass updates.]